Laborationsrapport

Projektuppgift / Väderapplikation
DT211G, Frontend-baserad webbutveckling

Författare: Alva Stern, alst2511@student.miun.se
Termin, år: VT, 2026

Mittuniversitetet
MID SWEDEN UNIVERSITY

1 Sammanfattning

I detta projekt har en webbapplikation som gör det möjligt för användare att söka efter väder på en plats, och även få upp den platsen på en karta skapats. Detta med syftet att det i många väderapplikationer inte är möjligt att kunna se vilken exakt plats som appliceras på vädret som visas, vilket kan bli problematiskt eftersom det finns många byar och orter med samma eller liknande namn. För att skapa webbapplikationen har en automatiserad arbetsprocess med Vite, CSS-preprocessorer och versionshantering med Git / Github använts.

2 Innehållsförteckning

1 Sammanfattning 1
2 Innehållsförteckning 2
3 Inledning 3
3.1 Problemmotivering 4
4 Bakgrundsmaterial 4
5 Metod 6
5.1 Desktop research 7
5.2 Wireframes 7
5.3 Fetch-anrop med async / await 8
5.4 SASS / SCSS 8
5.5 Keyframes och transition / transform 8
6 Lösningsalternativ 8
6.1 Skapande av idé och lösning 9
6.2 Användarupplevelse och grafisk profil 9
6.3 Automatiserad arbetsprocess med Vite 9
6.4 Versionshantering med Git / Github 10
6.5 Användning av fetch och async / await 10
6.6 Användning av plug-ins för att visa grafiskt material 10
6.7 Användning av SASS / SCSS 11
6.8 Animeringar 11
6.9 Optimering av webbapplikation 11
6.10 Automatiserad publicering 12
7 Resultat 12
8 Slutsatser 13
9 Källförteckning 14

3 Inledning

I denna projektuppgift hade jag som student en uppgift att skapa en webbapplikation som kombinerar data från flera olika JSON-baserade webbtjänster (API:er), detta genom användning av JavaScript, HTML och SCSS/SASS. Jag har baserat projektet på Node.js, använt mig av en automatisk arbetsprocess med hjälp av VITE, använt GIT och Github för versionshantering, samt använt lämpliga plug-ins för att visualisera mitt grafiska material på webbplatsen.

För att komma upp med en idé för vad min webbapplikation skulle ha för målgrupp och syfte genomfördes en session av brainstorming. Jag använde mig under denna session av desktop research för att ta reda på hur en webbapplikation skulle kunna byggas upp för att använda flera olika API:er på ett lämpligt sätt. Resultatet av detta blev en webbapplikation där en användare kan söka efter en plats, få upp den platsen på en karta, samt att vädret för den platsen visas på webbplatsen.

Målgruppen som jag identifierade är relativt stor eftersom väder är något som påverkar väldigt många människor. Min väderapplikation riktar sig därför till människor som är intresserade av att ta reda på vad det ska bli för väder under dagen eller de kommande dagarna.

3.1 Problemmotivering

När jag genomförde min session av brainstorming för att komma på en idé där flera olika API:er kan kombineras, kom jag relativt snabbt in på att visa väder. När jag genomförde djupare research om detta ämne noterade jag att det är väldigt få webbplatser som samtidigt som de visas väder, på ett tydligt sätt visar exakt vilken plats vädret appliceras på. Det enda som visas är namnet på orten/staden och dess område. Ett problem som jag därmed noterade är att det finns väldigt många namn på orter eller mindre byar som har samma namn som andra orter eller byar [1]. På grund av detta kan det ibland upplevas svårt för användare att med säkerhet veta att de ser vädret för exakt den plats som de vill.

För att bemöta detta problem kom jag upp med en idé om att samtidigt som en användare söker efter väder på en specifik plats, kommer platsen även att visas på en karta. Detta med syfte att användaren då kan försäkra sig om att det är rätt plats som visas. Genom detta skapas en tydlighet på webbplatsen, vilket är viktigt för att användare ska vilja stanna kvar på webbplatsen och besöka den oftare [2].

4 Bakgrundsmaterial

För att skapa denna webbplats användes olika tekniker för att skapa en automatiserad arbetsprocess, versionshantera och skapa en webbplats med information och innehåll från externa API:er. I denna del presenteras begrepp som kommer att förekomma i rapporten.

Automatiserad arbetsprocess: En automatiserad arbetsprocess inom detta område innebär att specifika programvaror eller teknologier används i arbetsprocessen med syfte att minimera tidskrävande, repetitiva eller regelbaserade uppgifter, detta med relativt liten mänsklig inblandning [3].

Vite: Vite är ett byggverktyg inom webbutveckling som har som syfte att skapa en snabbare och smidigare utvecklingsmiljö för moderna webbprojekt [4]. Vite erbjuder omfattande funktionsförbättringar och erbjuder möjligheten att bland annat paketera skriven kod med Rollup [4].

Versionshantering: Versionshantering innebär att tidigare versioner av ett dokument, källkodsfiler, program eller webbsidor sparas och därmed kan återskapas, samt att ändringar som gjorts i de tidigare versionerna kan spåras [5].

Github / GIT: Github är en molnbaserad plattform där användare kan lagra, dela och samarbeta kring kod [6]. Det är sedan GIT som är systemet för att spåra ändringar i olika filer, vilket underlättar i projekt där flera personer är med [6]. En utvecklare laddar alltså upp filer till Github som i sin tur sparas i GIT-repositorys, där kan GIT sedan automatiskt spåra ändringar i olika versioner [6].

API: Ett API (Applikations-Programmerings-Gränssnitt) är en specifikation av hur olika applikationsprogram kan kommunicera med en programvara, som exempelvis en webbapplikation [7]. Ett API utgörs ofta av ett externt länkat bibliotek som vid inkludering blir som en mjukvarokomponent på webbplatsen [7].

JSON: JSON (JavaScript Object Notation) är ett text-baserat format som lagrar och utbyter data mellan en server och en publiceringsplattform på ett sätt som är läsbart för både människor och datorer [8].

Fetch-anrop: Ett fetch-anrop är en metod inom programmering (främst JavaScript) som används för att hämta data från en specifik server via nätverket (ofta genom API:er) utan att behöva ladda om webbplatsen [9].

Grafisk profil: En grafisk profil är en samling med information som handlar om hur exempelvis ett företag eller organisation ser ut visuellt [10]. Den kan ofta utforma sig i en guide som visar på hur det visuella ska se ut och utformas för att bibehålla en enhetlighet i utseendet [10].

Wireframe: En wireframe är ett visuellt diagram som visar på hur skelettet för en digital plattform kan komma att se ut [11]. Den visar ofta på funktioner, hur olika element är kopplade till varande, navigationen och informationsarkitekturen i den digitala plattformen [10].

Prototyp: En prototyp är en första grundform eller testmodell av en specifik produkt [11]. Syftet med prototyper är att underlätta kommunikation och visualisering av olika idéer för att undersöka olika lösningar, samt upptäcka brister i produkten [11].

SASS / SCSS: Sass är en förkortning av Syntactically Awesome Style Sheets och är en populär CSS-preprocessor som erbjuder fler funktioner än vanlig CSS [12]. SASS består av två olika syntaxer, en mer traditionell och en som kallas för SCSS [12]. SCSS är en syntax som liknar traditionell CSS vilket gör att den blir enkelt för utvecklare att lära sig, samt blir enkel att läsa [12].

CSS-animation: CSS-animeringar är animationer på en digital plattform som kan skapa komplexa och dynamiska grafiska effekter på en webbplats [13]. Syftet med CSS-animation är att förbättra användares interaktiva upplevelse [13], genom att exempelvis få dem att titta på en specifik plats på den digitala plattformen som behöver användarens uppmärksamhet genom en rörelse på webbplatsen.

@keyframes: Keyframes är en CSS-regel som används för att skapa animationer och styr hur ett element ska förändras över tid [14]. Först definieras det som ska hända i början av animationen, sedan det som ska hända i mitten och till sist det som ska hända i slutet av animationen.

Transition / Transform: Transition och transform används i CSS/SASS för att skapa animationer och interaktiva effekter på webbplatser [15]. Transform justerar elementets utseende och position medan transition justerar övergångarna när elementet förändras genom animationen [15].

Branch: En branch är en arbetsyta inom ett projekt som är isolerad från resterande arbete i projektet [17]. Det innebär att utveckling och ändringar kan genomföras i en arbetsyta, medan det huvudsakliga materialet inte påverkas. När en utvecklare är tillfredsställd med arbete i exempelvis en development-branch, kan det arbetet förflyttas till en main-branch.

5 Metod

När jag genomförde denna projektuppgift använde jag mig utav flera olika metoder som underlättade för mig under arbetets gång. I denna del av rapporten presenteras och förklaras de olika metoderna som har använts.

5.1 Desktop research

Den första metoden jag använde mig av är desktop research. Desktop research går ut på att samla befintlig information, exempelvis via internet [16]. Denna metod använde jag mig av för att identifiera ett problem som kan lösas genom en webbapplikation som kombinerar data från olika API:er. Resultatet av detta blev att det är många väderapplikationer som inte låter användare se exakt den position som det presenterade vädret kan appliceras på, vilket blev det identifierade problem.

Fortsättningsvis använde jag mig av desktop research för att ta reda på hur väder brukar presenteras på digitala plattformar. Detta gjorde jag med syfte att kunna skapa min design med tagen inspiration från den design som redan är etablerad för att visa väder. Anledningen till detta är för att jag inte vill bryta mot några vanliga konventioner som användare är vana vid. Bryter en webbplats mot vanliga konventioner riskerar webbplatsen att inte bli så pass användarvänlig som jag vill.

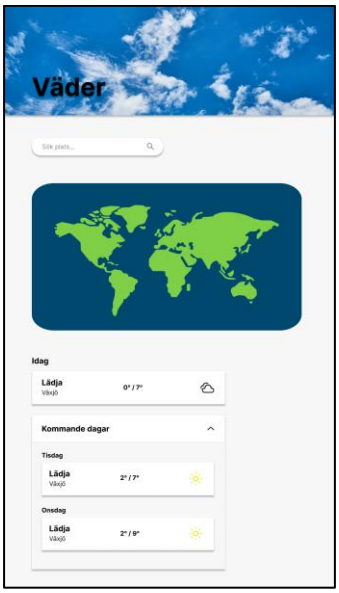
5.2 Wireframes

Efter att jag hade genomfört desktop research för att identifiera ett problem, samt ta reda på hur väder brukar presenteras på digitala plattformar, skapade jag en wireframe för webbplatsen, se figur 1. Detta gjorde jag med syfte att säkerställa att jag får med allt som behöver finnas på webbplatsen, se till att informationsarkitekturen fungerar, samt garantera en användarvänlig webbapplikation som inte bryter mot några typiska konventioner. Jag fick även en känsla över layouten och kunde utefter det hitta en lösning till hur jag kan inkludera en bild på min webbplats. Jag behöver en bild eftersom jag med användning av Vite ska optimera den med en automatisk arbetsprocess.

Figur 1

Efter att jag hade skapat min första wireframe skapade jag en ny version där jag istället för att fokusera på informationsarkitektur och layout, fokuserade på det visuella och den grafiska profilen, se figur 2. Detta gjorde jag med syfte att få en känsla för webbplatsens

grafiska profil, samt se till att webbapplikationen blir visuellt snygg att titta på för användare.



Figur 2

5.3 Fetch-anrop med async / await

För att hämta data från externa servrar (API:er) till min webbapplikation använde jag mig av fetch-anrop tillsammans med async/await. Jag använde mig at fetch eftersom det är en lämplig metod för att be webbapplikationen skicka förfrågningar till andra servrar som i sin tur skickar tillbaka data i form av exempelvis JSON, som kan omvandlas och bli till innehåll på webbplatsen. Async/await användes som metod för detta eftersom det kan hantera asynkrona fetch-anrop på ett sätt som gör koden lättläst, samt på ett sätt som gör att API:et väntar på svar innan datan används i webbapplikationen.

5.4 SASS / SCSS

För att styla den framtagna webbapplikationen användes SASS/SCSS som metod och tillägg till traditionell CSS. Detta genomfördes eftersom denna CSS-preprocessor har funktioner som effektiviserar och förenklar arbetet och därmed underlättar arbetet med stylingen. Funktioner inom SASS/SCSS som användes under stylingen är för det första partials som möjliggör en bättre struktur i filerna, samt vairabler och mixins för att undvika repetitiv kod.

5.5 Keyframes och transition / transform

Keyframes och transition/transform användes som metod för att skapa grafiska effekter och animeringar i webbapplikationen. Keyframes användes med syfte att definiera hur animationer ska förändra ett element över tid, vad som ska hända i början, i mitten och i slutet av animationen. Transition och transform användes för att styra olika elements storlek, position eller rotation vid olika interaktioner som exempelvis hover. Genom användning av dessa metoder skapas en mer dynamisk och interaktiv användarupplevelse.

6 Lösningsalternativ

I denna del av rapporten beskrivs hur projektet har genomförts i praktiken, vilka metoder som har använts och hur problemet har löst med hjälp av dessa metoder.

6.1 Skapande av idé och lösning

För att komma upp med en lösning till det identifierade problemet som är att många väderapplikationer inte på ett tydligt sätt visar exakt vilken plats vädret kan appliceras på, använde jag mig som tidigare nämnt av en brainstorming-session. Under den sessionen kom jag upp med lösningar som handlar om att användare förutom att se vädret, ska få upp platsen på en karta för att tydligare se vilken plats som vädret som visas på. Denna lösning är lämplig för denna uppgift eftersom den ger möjlighet för användning av flera API:er som samarbetar med varandra i webbapplikationen.

6.2 Användarupplevelse och grafisk profil

För att sedan utveckla lösningen i praktiken började jag med att skapa en wireframe över webbapplikationen, se figur 1. Detta gjordes med syftet att säkerställa att alla obligatoriska delar av webbapplikationen kommer med, samt för att ge fokus åt användarvänligheten, informationsarkitekturen och upplevelsen. Detta är av stor vikt eftersom användare tenderar att vilja stanna kvar, samt återkomma till webbplatsen om den upplevs som tydlig, enkel att använda och visuellt tilltalande [2].

Efter att jag hade skapat wireframes och haft fokus på användarupplevelsen gick jag över till att ta fram en grafisk profil och därmed den visuella delen. Eftersom tanken med webbapplikationen är att användare på ett så enkelt och snabbt sätt som möjligt ta reda på vad det är för väder, skapades en enkel och professionell grafisk profil. Den grafiska profilen skapades genom att det togs fram en ny wireframe, baserad på den tidigare skapade wireframes, fast där fokus istället blev att ta fram en passande grafisk profil för ändamålet. Vid framtagandet blev resultatet en enkel och ljus grafisk profil med inslag av blåa färger i en enkel bild, för målet om en professionell och enkel grafisk profil, se figur 2.

6.3 Automatiserad arbetsprocess med Vite

För att utveckla webbapplikationen användes en automatiserad arbetsprocess med hjälp av Vite. En automatiserad arbetsprocess användes med syfte att underlätta utvecklingen av webbapplikationen genom att undvika repetitivt och tidskrävande arbete. För att skapa en automatiserad arbetsmiljö användes Vite som är ett byggverktyg vars funktioner och funktionsförbättringar kan skapa en snabb och smidig utvecklingsmiljö för webbprojekt.

För att skapa ett projekt med Vite som byggverktyg användes till att börja med terminalen för en lämplig mapp för projektet. I terminalen skrevs *npm create vite@latest Projektuppgift,* vilket skapade ett Vite-projektet med namnet *Projektuppgift.* Fortsättningsvis användes Vite under hela projektet genom funktioner som att visa webbapplikationens utveckling på en live server. Jag som utvecklare fick tillgång till denna server genom att i terminalen för projektet skriva *npm run dev*, vilket gav mig en länk till live-servern.

Sedan användes Vite för att på ett automatiserat sätt optimera grafiskt material i form av bilder i webbapplikationen. För att genomföra detta installerades först en plug-in för att göra detta möjligt, i terminalen skrev då *npm install vite-plugin-imagemin –save-dev.* Fortsättningsvis importerades plug-in:en i filen *vite.config.js* där kvalitén på bilderna vid

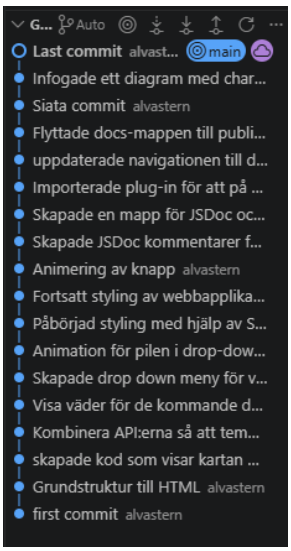
optimering definierades. Bilderna optimerades sedan när filerna minifierades med *npm run build*.

Vite användes även för att optimera alla filer som sedan skulle publiceras till servern. För att optimera filerna och göra dem redo för publicering skrevs *npm run build* in i terminalen för projektet. När detta gjordes optimerades automatiskt alla filer och lades i en mapp som heter dist, vilket är en mapp som Vite automatiskt skapar och där alla optimerade filer hamnar. Det som händer när filerna optimeras är att de minifieras genom att exempelvis kommentarer och onödiga mellanslag tas bort. Anledningen till att filerna ska minifieras och vara så små som möjligt är att de då kräver mindre prestanda för att laddas in på webben.

6.4 Versionshantering med Git / Github

Under projektets gång har alla filer versionshanterats kontinuerligt. Det innebär att tidigare versioner av ett dokument eller källkodsfiler sparas och kan återskapas, samt att ändringar kan spåras [5]. I detta projekt användes verktygen Git och Github för versionshantering. Github lagrar dokument och källkod i så kallade repositorys där sedan Git kan används för att spåra ändringar i olika filer [6].

För att versionshantera denna projektuppgift skapades till att börja med två olika branches, main och development. Därefter skapade jag på ett automatiskt sätt ett Git-repository genom att publicera projektet till Github. Fortsättningsvis utvecklade jag webbapplikationen i development-branchen och skapade olika commits för att spara och lagra olika versioner av källkoden, se figur 3. Detta med syfte att kunna gå tillbaka och spåra ändringar. En version av koden sparas genom att göra en commit med ett meddelande till Github via Git. När arbetet i development-branchen var färdigställt pushades det till main-branchen, vilket tydliggör att koden som finns i main-branchen är färdigställt arbete, och kodfiler som ligger i development-branchen, är under utveckling och inte färdigställt.



Figur 3

6.5 Användning av fetch och async / await

För att hämta data från de externa API:erna och inkludera den i webbapplikationen användes fetch och async/await. Dessa användes med syfte att möjliggöra hantering av asynkrona fetch-anrop som skapar en lättläst kod. Fetch och async/await användes på det sätt att när en användare söker efter en plats i webbapplikationen, skickas till att börja med ett fetch-anrop till Nominatim API, vilket är en API som returnerar koordinater till den sökta platsen. Dessa koordinater användes sedan för att visa platsen på en karta, samt hämta väderdata från Open-Meteo API.

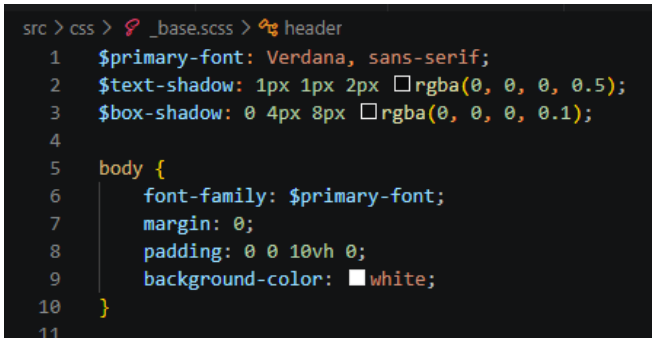
För att hantera nätverksanropen användes async/await, vilket gör det möjligt att skriva synkron kod på ett lättläst sätt. Genom användning av await kan webbapplikationen därmed vänta på att svar från API:erna returneras innan datan visas i webbapplikationen.

6.6 Användning av plug-ins för att visa grafiskt material

För att inkludera grafiskt material som är infogat med hjälp av plug-ins, inkluderades ett linjediagram som visar på hur temperaturen har förändrats över en dag på en plats som är vald av användaren. Den plug-in som har använts för att infoga detta grafiska material är biblioteket chart.js. För att inkludera diagrammet installerades till att börja med chart.js till projektet, vilket gjordes genom att i terminalen skriva *npm install chart.js*. Därefter importerades chart.js till JS-filen och inkluderades genom ett objekt som skapades från en klass med hjälp av syntaxen New.

6.7 Användning av SASS / SCSS

För att styla webbapplikationen användes CSS-preprocessorerna SASS/SCSS. Dessa användes eftersom detta tillägg bidrar med flera användbara funktioner som underlättar arbetet med stylingen och skapar en bättre struktur i filerna för styling. Den första funktionen som användes är partials som möjliggör för utvecklare att dela upp koden för styling i olika filer och därmed skapa en bättre och tydligare struktur. Partials som användes i detta projekt är *base, components, animations, map och layout* och importerades med hjälp av @use till en main scss-fil, som i sin tur var importerad till main.js.



Den andra och tredje funktionerna i SASS/SCSS som användes i denna projektuppgift är variabler och mixins. Båda dessa funktioner minimerar risken för repetitiv kod och bildar därmed kodfiler som tar mindre plats och därmed är optimerade på ett stabilare sätt. Variabler användes genom att definiera en variabel och dess värde på exempelvis detta vis, *$primary-color: white;*. Denna variabel används sedan varje gång en text skulle vara vit. Istället för att varje gång skriva *color: white;,* skrevs istället *color: $primary-color,* vilket skapar en icke-repetitiv kod, se figur ovan.

Mixins minimerade också risken för repetitiv kod eftersom ett stycke kod kan användas på flera platser i koden. Ett exempel på en mixin är att det först identifieras att det är en mixin som skapas, samt ett namn på den, *@mixin väder-kort.* Under denna mixin kan sedan specifik styling definieras. För att sedan använda den kod som ligger i den skapade mixin används *@include,* när den används i en regeluppsättning för CSS inkluderas alla selektorer och egenskaper som definierades i *@mixin väder-kort*. Genom detta blir koden mindre repetitiv.

6.8 Animeringar

I denna webbapplikation har två animeringar skapats. Den första animeringen som skapades skapar en färgändring och liten rotation på sök-knappen när en användare hovrar över den. Detta genomfördes med användning av @keyframes, CSS transition och CSS transform, där först keyframes och transform användes för att ge knappen tre olika roteringsgrader, en vid 0%, en vid 50% och en vid 100%. Sedan användes transition och den definierade keyframe för att skapa en mjuk övergång, både för roteringen och färgskiftet när användaren hovrar över knappen. Syftet med denna animation är att användaren blir medveten om att knappen är möjligt att trycka på, samt att de kan trycka på knappen för att söka efter en plats på kartan.

Den andra animeringen som skapades innebär att pilen som är i sektionen för att klicka ner vädret för de kommande dagarna, roterar 180° när användaren klickar på den. Detta gjordes genom användning av transition, transform och en funktion i javascript, där transform definierar att pilen ska rotera 180°, transition beskriver under vilken tid animationen ska ske, samt vad den ska ha för övergång, och javascript styr att animationen ska köras när användaren klickar på pilen. Syftet med denna animation är att tydliggöra att sektionen fungerar som en drop-down meny och därmed går att klicka fram och klicka bort. Genom att pilen roterar och är i en specifik riktning beroende på om filen är nerfälld eller inte, skapas en tydlighet för användaren om flikens status.

6.9 Optimering av webbapplikation

För att optimera webbplatsen användes tre olika tekniker. Den första teknik som användes som tidigare nämnt för att på ett automatiskt sätt optimera bilder. Då användes en installerad plug-in som heter *imagemin,* för att på så sätt möjliggöra automatisk optimering av bilder. Kvalitén som bilderna skulle ha efter optimering definierades i *vite.config.js,* sedan blev bilderna optimerade när filerna minifierades genom *npm run build*.

Den andra teknik som användes var att alla filer optimerades så mycket som möjligt genom *npm run build.* När *npm run build* körs minifieras alla JavaScript-filer och CSS-filer, vilket innebär att de blir så små som möjligt och därmed optimeras, samt blir till filer som kräver mindre prestanda för att laddas in på webben. Genom *npm run build* omvandlas dessutom alla SASS/SCSS-filer till CSS-filer eftersom webbläsare inte kan läsa av SASS/SCSS-filer.

Den tredje tekniken som användes för att optimera webbapplikationen var att fetch-anropen endast genomförs om en användare söker efter data som hämtas från externa API:er. Hade fetch-anropen genomförts när webbapplikationen läses in, hade det blivit mer för webbläsaren att läsa in vilket kan resultera i att webbplatsen kräver mer tid för att läsas in och därmed bli långsam. Därmed optimeras webbplatsen genom att externa data endast läses in om användaren begär det.

6.10 Automatiserad publicering

Efter att alla filer blivit minifierade med *npm run build,* kontrollerade filerna med *npm run preview* och webbapplikationen var färdigställd, publicerades den automatiskt till Netlify. Till att börja med bytte jag branch från development till main, sedan mergades alla filer som det arbetats med i development till main. Därefter användes Netlify för att importera projektet, vilket resulterade i att projektet direkt blev publicerat.

7 Resultat

Denna projektuppgift resulterade i en webbapplikation som gör det möjligt för användare att ta reda på väder och få upp platsen där vädret visas på en karta, samt möjligheten att se dagens temperatur i ett linjediagram. Linjediagrammet framkom under utvecklingens gång då en plug-in av något slag skulle användas i projektuppgiften. Det är möjligt för användaren att se vädret för idag, de tre kommande dagarna, samt se vädret över några dagar tillbaka över tid i ett diagram. Det identifierade problemet är att det kan upplevas som utmanande för användare att med säkerhet veta att det väder som visas i webbapplikationen är för den plats de har sökt efter, detta eftersom det är många byar och orter som har samma eller liknande namn [1]. Lösningen bemöter alltså detta problem genom att visa platsen som användaren söker efter även på en karta och inte endast i textformat.

8 Slutsatser

Utifrån denna projektuppgift har jag kommit till insikter angående vad som gick bra i projektet och vad som hade kunnat göras bättre eller annorlunda. Det som jag anser fungerade bra i detta projekt var främst arbetet med de nya teknikerna som har presenterats i denna kurs. Att arbeta med en automatiserad arbetsprocess med hjälp av Vite har för mig fungerar på ett utmärkt sätt. Jag anser att det finns flera funktioner inom detta som underlättar arbetet med utveckling av webbplatser och webbapplikationer. En funktion inom detta som jag tycker extra mycket om är möjligheten till att använda *npm run build* och därmed på ett automatiserat sätt optimera filer, samt omvandla SASS/SCSS-kod till traditionell CSS. Detta tycker jag är en mycket bra och effektiv teknik för att göra filer så små som möjligt och därmed kunna laddas in snabbare i webbläsare och genom det, förbättra upplevelsen för användarna på webbplatsen / webbapplikationen.

Något som jag tycker var utmanande med denna projektuppgift var att hitta ett lämpligt och relevant problem som den utvecklade webbapplikationen på något sätt skulle lösa. När jag genomförde uppgiften började jag med att identifiera en lämplig webbapplikation som skulle kunna innehålla alla de krav och riktlinjer som uppgiften är baserad på. Därefter identifierade jag ett problem som min tänkta webbapplikation skulle lösa, vilket var väldigt utmanande. Det som jag därmed hade kunnat göra annorlunda är att istället för att börja med att hitta en idé till webbapplikation som är genomförbar med uppgiftens riktlinjer och krav, börja med att identifiera ett problem som kan lösas med någon typ av webbplats eller webbapplikation.

9 Källförteckning

[1] Språkbruk. Identiska ortnamn – preciserade adressnamn [internet]. 2012-10-02.
https://sprakbruk.fi/artiklar/identiska-ortnamn-preciserade-adressnamn/
Hämtad 2026-03-07.

[2] GOWEB. Fördelar med en användarvänlig webbdesign [internet]. N/D.
https://goweb.se/news/view/fordelar-med-en-anvandarvanlig-webbdesign/#:~:text=Webbdesign%20%C3%A4r%20en%20avg%C3%B6rande%20faktor%20f%C3%B6r%20hur,och%20konverteringar%20samtidigt%20som%20det%20minskar%20avvisningsfrekvensen.
Hämatd 2026-03-07.

[3] SAP. What is process automation? [internet]. N/D.
https://www.sap.com/products/technology-platform/process-automation/what-is-processautomation.html#:~:text=Process%20automation%20definition-,Process%20automation%20is%20defined%20as%20the%20use%20of%20software%20and,employee%2C%20or%20providing%20customer%20service.
Hämtad 2026-03-07.

[4] Vite. Komma igång [internet]. N/D.
https://vite.dev/guide/
Hämtad 2026-03-07.

[5] Wikipedia. Versionshantering [internet]. N/D.
https://sv.wikipedia.org/wiki/Versionshantering
Hämtad 2026-03-07.

[6] Github Docs. About Github and Git [internet]. N/D.
https://docs.github.com/en/get-started/start-your-journey/about-github-and-git
Hämatd 2026-03-07.

[7] Wikipedia. Applikationsprogrammeringsgränssnitt [internet]. N/D.
https://sv.wikipedia.org/wiki/Applikationsprogrammeringsgr%C3%A4nssnitt
Hämtad 2026-03-07.

[8] Oracle. Vad är JSON? [internet]. 2024-03-04.
https://www.oracle.com/database/what-is-json/
Hämtad 2026-03-09.

[9] Lenovo. What is fetch? [internet]. N/D.
https://www.lenovo.com/in/en/glossary/what-is-fetch/?orgRef=https%253A%252F%252Fwww.google.com%252F
Hämtad 2026-03-09.

[10] Något. Grafisk profil – vad är det och vad innehåller den? [internet]. 2021-05-12.
https://nagotkommunikation.se/vad-ar-en-grafisk-profil/
Hämtad 2026-03-09.

[11] Wikipedia. Prototyp [internet]. N/D.
https://sv.wikipedia.org/wiki/Prototyp
Hämtad 2026-03-09.

[12] Scaler Academy. SASS vs SCSS: Vad är skillnaden? [internet]. 2023-09-01.
https://www.interviewbit.com/blog/sass-vs-scss/
Hämtad 2026-03-09.

[13] Condefinity. Vad är CSS-animationer? [internet]. N/D.
https://codefinity.com/sv/courses/v2/cb1e9565-bae9-4822-b190-52a6cb19f4cc/1aac733c-c039-48b0-a7be-5726b0ae9bd1/add9af23-c81d-436c-9e10-f72649ed9c44
Hämtad 2026-03-09.

[14] MDN. @keyframes [internet]. 2026-01-07.
https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@keyframes
Hämtad 2026-03-09.

[15] MDN. Using CSS transitions [internet]. N/D.
https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Transitions/Using
Hämtad 2026-03-09.

[16] FIRMAO. Skrivbordsforskning: vad är det? [internet]. 2022-08-14
https://se.firmao.io/blog_net/se/marketing/desk-research-what-is-it
Hämtad 2026-03-09.

[17] The Infosphere. Vad är Repository, Branch och Commit? [internet]. N/D.
https://info.ssis.nu/books/git/page/vad-ar-repository-branch-och-commit
Hämtad 2026-03-12.